SECTION 1 · SUBSECTION 1.1
SCOPE AND READING MAP

1.1.1 INTRODUCTION.

This handbook is the systems-data volume for the Project Gnosis code-archaeology system. It describes what the system is, how its nine subsystems work, the order in which they execute, every file that belongs to the repository, and the decisions that shaped them. The single idea the reader must leave with is that Gnosis is a deterministic seven-stage compiler wrapped around one hosted language-model call, not an autonomous agent swarm (figure 1.1-1).

The document is organised for three readers. An operator starts at section 7 and uses section 3 for fault isolation. A reviewer reads sections 1, 4 and 6. A maintainer reads section 3 in pipeline order, then section 5 for the file inventory. Section 8 records defects and unresolved questions so that no reader mistakes a known gap for an undocumented feature.

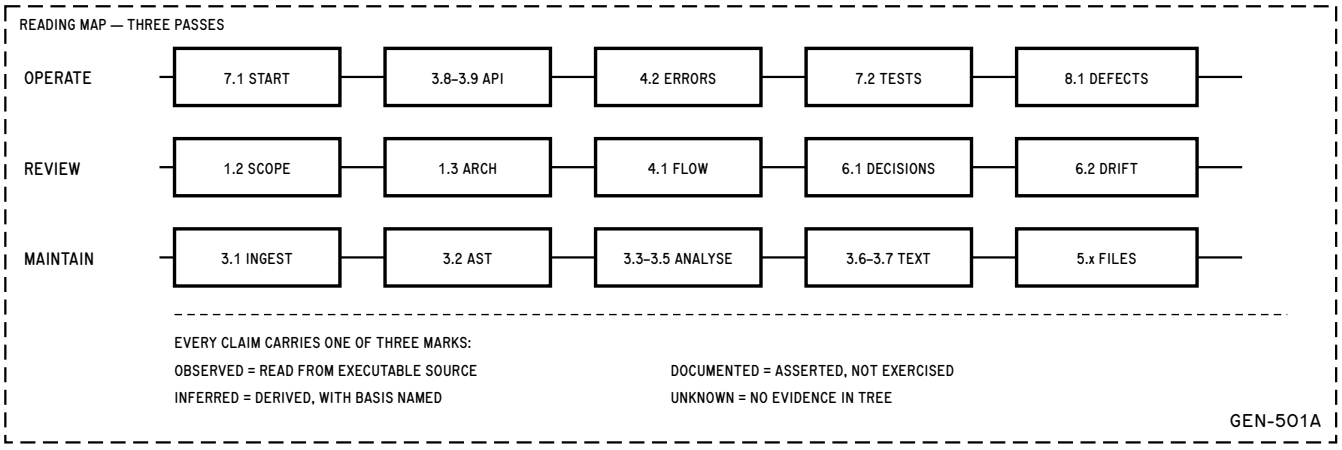


Figure 1.1-1. Reading Map and Evidence Marks

1.1.2 HOW TO USE THIS DOCUMENT.

The page number is the address. A reference such as 3.5-1 means section 3, subsection 3.5, sheet 1. Figure and table numbers run continuously inside a subsection. Paragraph numbers are stable and are used in the source-verification appendix.

Three marks qualify evidence. **OBSERVED** statements were read from source, configuration or test code in this working tree. **DOCUMENTED** statements come from a repository prose file, comment or commit message and were not exercised. **INFERRED** statements are conclusions; the basis is named in the sentence. A claim that cannot be placed in one of these classes is listed as an open item in 8.2-1.

1.1.3 SECTION INDEX.

Table 1.1-1. What Each Section Answers

Section	Question answered
1.2	What problem is solved, and what is explicitly outside the boundary?
1.3	What are the parts, and how are they deployed?
2.1	Which software is declared, which version actually resolves, and what does it do?
3.1–3.7	How does each pipeline stage transform state?
3.8–3.9	How is work submitted, stored, queued and displayed?
4.1–4.2	In what order does execution occur, and what happens on failure?
5.1–5.2	What is every file for, and what is deliberately excluded?
6.1–6.2	Why was each choice made, and where do documents disagree with code?
7.1–7.2	How is the system started, and what has actually been tested?
8.1–8.2	What is broken, limited, simplified or still unknown?

1.1.4 Reading Sequences.

Table 1.1-2. Suggested Reading Sequences

Reader	Order
Newcomer	1.1 → 1.2 → 1.3 → 4.1 → 2.1-1
Maintainer	3.1 → 3.2 → 3.3–3.5 → 3.6–3.9 → 4.1 → 5.1-1
Reviewer	1.3 → 6.1 → 6.2 → 8.1 → 8.2
Operator	7.1 → 3.8-1 → 4.2 → 7.2 → 8.1
Verifier	8.2 → cited file → test → verdict

CAUTION

Repository prose is stale in several places. The README still describes a sequential pipeline, React 18, an in-memory-only job store, an *azure-deploy.yml* workflow and 407 tests. The code and the test run show otherwise. Read 6.2-1 before quoting any document as fact.

FIRST PASS	ANY READER

```
1 1.2 - read the boundary
2 1.3 - trace figure 1.3-1
3 4.1 - follow one job
  √ STATE - seven stages, one store
4 3.x - open the stage in question
5 6.2 - check the drift ledger
6 8.1 - read known defects
```

GEN